Inside the proofs

Recognition, retrieval and confident errors, in Lean

(partner to "Recognition, retrieval and confident errors")

Jeff Shrager

October 2026

The proof, layer by layer

`MultiDigit/AnyMemory`, `Experiments/FaceNaming`: instances

`Phenomena/RetrievalLaws`: laws; `confident_errors_of_laws`

`Process/RecognizeRetrieve`: the composition; `confident_errors`

`Process/SampledRetrieval`: DOA search; closed forms $(1 - R)^n$, $Q \cdot G_n$

`Memory/DiscriminationNet`: nets, `select`, `path`; Lemma A (confusions agree on the path)

`Rep/Distribution`: exact finite distributions; `bind`, `prob`, the two-stage lemma

In words: Exact distributions

- A **distribution** is a finite list of outcomes, each with an exact rational probability.
- **Certain**: the distribution certain of *a* has the single outcome *a*, with probability 1.
- **Two stages** (*bind*): draw *a* from the first distribution, then draw from a second distribution that depends on *a*. Each second-stage probability is multiplied by the probability of the *a* that led to it.
- **Probability of an event**: add up the probabilities of the outcomes in the event.

Exact distributions

```
abbrev Dist (α : Type) := List (α × Rat)

/-- The distribution certain of `a`. -/
def pure (a : α) : Dist α := [(a, 1)]

/-- A two-stage process: draw `a` from `d`, then draw from `f a`. -/
def bind (d : Dist α) (f : α → Dist β) : Dist β :=
  match d with
  | [] => []
  | (a, p) :: d => ((f a).map fun (b, q) => (b, p * q)) ++ bind d f

/-- The probability of an event. -/
def prob : Dist α → (α → Bool) → Rat
  | [], _ => 0
  | (a, p) :: d, e => (if e a then p else 0) + prob d e
```

In words: The two-stage lemma

Lemma (two stages). The probability of an event after a two-stage process is the sum, over first-stage outcomes, of each outcome's probability times the event's probability after it.

Proof, by induction on the list of first-stage outcomes.

- No outcomes: both sides are 0.
- One more outcome: the event's probability in that outcome's branch, scaled by the outcome's probability, plus the rest, which is the induction hypothesis.

Two special cases are used throughout: relabelling the outcomes relabels the event; and a second stage that is certain on one set of outcomes and has a fixed probability on another.

The two-stage lemma

```
/-- A mixture: the probability of an event after a two-stage process is the
sum, over first-stage outcomes, of each one's probability times the event's
probability after it. -/
theorem prob_bind_sum (d : Dist α) (f : α → Dist β) (e : β → Bool) :
    prob (bind d f) e = (d.map fun ap => ap.2 * prob (f ap.1) e).sum := by
  induction d with
  | nil => simp [bind, prob]
  | cons x d ih => obtain ⟨a, p⟩ := x; rw [prob_bind_cons, ih]; simp
```

Its specializations do most of the work above:

- `prob_bind_pure`: relabelling outcomes relabels the event;
- `prob_bind_split`: a second stage that is certain on one set of outcomes and constant on another.

In words: Discrimination nets

- A **net** is either a terminal, holding a stored image or nothing, or a test on one feature with two subnets.
- **Selecting** a code: at each test, follow the branch the code's feature says; return the image at the terminal reached.
- The **path** of a code: the features tested on the way down.
- A net **recognizes** a code when the code selects its own image.

Discrimination nets

```
inductive Net where
  | leaf : Option Item → Net
  | test : Feature → Net → Net → Net

/-- Selection: sort a code to a terminal and return its image. -/
def select : Net → Item → Option Item
  | leaf img, _ => img
  | test f n0 n1, x => if x.feat f then n1.select x else n0.select x

/-- The features examined while sorting `x`: the path it takes. -/
def path : Net → Item → List Feature
  | leaf _, _ => []
  | test f n0 n1, x => f :: (if x.feat f then n1.path x else n0.path x)
```

In words: Lemma A

Lemma A. Suppose each image is stored only once, the net recognizes y, and x selects y's image. Then x agrees with y on every feature tested along x's path.

Proof, by induction on the net.

- At a terminal there is no path, so there is nothing to show.
- At a test, x and y each go one way.
  - The same way: they agree on the tested feature, and the rest of the path is the induction hypothesis.
  - Different ways: y's image would be selected on both sides, so it would be stored twice. Contradiction.

Lemma A: a confusion agrees on the path

```
theorem confusion_agrees_on_path (N : Net) (x y : Item) (hnd : N.images.Nodup)
    (hy : N.select y = some y) (hx : N.select x = some y) :
    ∀ f ∈ N.path x, x.feat f = y.feat f := by
  induction N with
  | leaf img => simp [path]
  | test f n0 n1 ih0 ih1 =>
    -- ...
    cases hxf : x.feat f <;> cases hyf : y.feat f <;> simp [hxf, hyf] at hx hy hg
    · -- both go to n0: the tested feature agrees, recurse
      rcases hg with rfl | hg
      · rw [hxf, hyf]
      · exact ih0 hnd0 hy hx g hg
    · -- x to n0, y to n1: y would be stored in both subtrees
      exact absurd rfl (hdisj y (select_mem_images n0 x y hx) y (select_mem_images n1 y y
      ↪ hy))
    -- ... (the two symmetric cases)
```

In words: DOA search, and a closed form

Search with n tries: sample an answer in proportion to strength. If it clears the criterion, state it; otherwise search again with one try fewer. With no tries left, retrieval is exhausted.

Theorem. If the total strength is not 0, retrieval is exhausted with probability $(1 - R)^n$, where R is the chance that one sample clears the criterion.

Proof, by induction on n.

- No tries: exhausted for certain, and $(1 - R)^0 = 1$.
- One more try: a sample that clears the criterion ends the search, which is then not exhausted. One that doesn't (probability $1 - R$) leads to a search exhausted with probability $(1 - R)^n$, by induction. So the total is $(1 - R) \cdot (1 - R)^n$.

DOA search, and a closed form

```
def search (as : List α) (s : α → Rat) (cc : Rat) : Nat → Nat → Dist (Retrieval α)
  | _, 0 => Dist.pure .exhausted
  | k, n + 1 => Dist.bind (sample as s) fun a =>
      if above s cc a then Dist.pure (.stated a (k + 1)) else search as s cc (k + 1) n

theorem prob_exhausted (as : List α) (s : α → Rat) (cc : Rat) (hT : total as s ≠ 0) :
    ∀ n k, prob (search as s cc k n) Retrieval.isExhausted = (1 - reach as s cc) ^ n := by
  intro n
  induction n with
  | zero => intro k; simp [search, prob_pure, Retrieval.isExhausted]
  | succ n ih =>
    intro k
    simp only [search]
    rw [prob_bind_split _ _ _ (fun _ => false) (fun a => !above s cc a) ((1 - reach as s
    ↪  cc) ^ n)]
    · rw [prob_false, prob_not_above as s cc hT, Rat.pow_succ]; grind
    · intro a
      by_cases ha : above s cc a
      · simp [ha, prob_pure, Retrieval.isExhausted]; grind
      · simp [ha, ih]; grind
```

In words: The composition

After recognition:

- if memory calls up nothing, use the backup;
- if it calls up a stored code y, search y's associations. A stated answer is a *whole-problem* answer; if the search is exhausted, use the backup.

Recognize, then retrieve, else back up: a two-stage process. First, what memory calls up for the problem's code (a distribution); then the step above.

Nothing here mentions arithmetic, EPAM, or ACT-R: the problems, the answers and the memory are all parameters.

The composition

```
def afterRecognition (str : Item → A → Rat) (as : List A) (cc : Rat) (n : Nat)
    (backup : P → Dist A) (x : P) : Option Item → Dist (Route × A)
  | none => Dist.bind (backup x) fun r => Dist.pure (.backup, r)
  | some y => Dist.bind (search as (str y) cc 0 n) fun
      | .stated a _ => Dist.pure (.whole, a)
      | .exhausted => Dist.bind (backup x) fun r => Dist.pure (.backup, r)

/-- Recognize, then retrieve, else back up. -/
def recognizeRetrieve (code : P → Item) (retrieve : Item → Dist (Option Item))
    (str : Item → A → Rat) (as : List A) (cc : Rat) (n : Nat) (backup : P → Dist A) (x
    ↪  : P) :
    Dist (Route × A) :=
  Dist.bind (retrieve (code x)) (RR.afterRecognition str as cc n backup x)
```

Generic in the problem type `P`, the answer type `A`, and the memory (`retrieve`).

In words: Confident errors: the proof is a composition

Theorem (confident errors). Suppose:

- a familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.

Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```

In words: One supporting lemma

Lemma (the neighbour's answer). If x calls up q's code for certain, and q's associations are sharp, then x is never given a whole answer other than q's correct answer.

Proof.

1. Because x's recognition is certain, x's whole answers come from searching q's associations.
2. The chance that a search states an answer with some property is (the chance that one sample clears the criterion and has the property) times a geometric factor.
3. No answer other than q's clears the criterion, so that chance is 0.

One supporting lemma

```
theorem confusion_gives_neighbours_answer (correct : P → A) (x q : P)
    (hT : total as (str (code q)) ≠ 0) (h : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) :
    prob (recognizeRetrieve code retrieve str as cc n backup x)
      (wholeSuch fun a => a != correct q) = 0 := by
  rw [whole_depends_only_on_route code retrieve str as cc n backup x _ _ h,
    prob_stated _ _ _ _ hT]
  have : reachSuch as (str (code q)) cc (fun a => a != correct q) = 0 := by
    simp only [reachSuch]
    rw [prob_congr _ _ (fun _ => false), prob_false]
    intro a
    by_cases ha : above (str (code q)) cc a
    · simp [hs a ha]
    · simp [ha]
  rw [this]; grind
```

In words: Memory as an interface

- A **memory**: studying an item changes its state; retrieving with a probe gives a distribution over stored items, or nothing.
- **EPAM's net** as a memory: a probe calls up the image it selects, for certain.
- **ACT-R's declarative memory** (decay 1, temperature 1): a probe calls up each chunk with probability proportional to its weight. A chunk's weight is the sum, over its uses, of $1/\text{age}$, times μ for each mismatching feature. Failure always has weight θ.
- **Theorem.** ACT-R never retrieves anything for certain, because failure is always among the outcomes.

Memory as an interface

```
structure Retriever (State Stim : Type) where
  study : State → Stim → State
  retrieve : State → Stim → Dist (Option Stim)

-- EPAM: what a net's probe calls up, for certain
def Net.retrieve (N : Net) : Item → Dist (Option Item) := fun y => Dist.pure (N.select y)
def netRetriever (order : NoticingOrder) : Retriever Net Item := (netLearner
↪  order).toRetriever

-- ACT-R (decay 1, temperature 1): sample in proportion to weight, failure included
def retrieve (P : ActParams) (s : ActState) (x : Item) : Dist (Option Item) :=
  Dist.sample (outcomes s) (weight P s x)

/-- Retrieval is never certain. Failure is always among the outcomes. -/
theorem retrieve_not_certain (P : ActParams) (s : ActState) (x y : Item) :
    retrieve P s x ≠ Dist.pure (some y) := by
  intro h
  simp [retrieve, Dist.sample, outcomes, Dist.pure] at h
```

In words: The bridge to multi-digit addition

- The multi-digit model keeps its own names: the backup is the *column* route.
- **Relabelling** translates the general composition's routes into the model's.
- **Lemma (the bridge).** Every probability of a multi-digit trial equals the probability of the relabelled event in the general composition.
- **Proof**: expand both sides with the two-stage lemma, and compare them outcome by outcome.
- So each multi-digit theorem is: rewrite with the bridge, then apply the general theorem.

The bridge to multi-digit addition

```
/-- The generic routes, in this model's names. -/
def relabel : Cog.Route × Nat → Route × Nat
  | (.whole, a) => (.whole, a)
  | (.backup, a) => (.columns, a)

theorem prob_trialWith (x : Problem) (e : Route × Nat → Bool) :
    prob (trialWith retrieve E str as cc n backup x) e =
      prob (recognizeRetrieve E.code retrieve str as cc n backup x) (fun o => e (relabel
      ↪  o)) := by
  unfold trialWith recognizeRetrieve
  rw [prob_bind_sum, prob_bind_sum]
  congr 1
  apply List.map_congr_left
  intro rp _
  rw [prob_after]

-- then each multi-digit theorem is two lines:
theorem confident_errors ... := by
  simp only [prob_trialWith, wholeWrong_relabel, isWhole_relabel, isColumns_relabel]
  exact RR.confident_errors E.code retrieve str as cc n backup Problem.sum x q ...
```

In words: Laws

The **laws** quantify over the states reachable from a starting state by studying items from a set U.

- **Certainty**: an item just studied retrieves itself with probability 1.
- **Stability**: studying a different item never changes the chance that an item retrieves itself.
- **Determinacy**: every retrieval has a single outcome.
- **Lemma.** Under determinacy, a positive chance of retrieving y means y is retrieved for certain; so does a chance of 1.

Laws

```
def Certainty (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ y ∈ U, pRet R (R.study s y) y y = 1

def Stability (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ y ∈ U, ∀ z ∈ U, z ≠ y → pRet R (R.study s z) y y = pRet R
  ↪  s y y

def Determinate (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ x, ∃ r, R.retrieve s x = Dist.pure r

/-- Under determinacy, a positive chance of retrieving `y` is certainty. -/
theorem retrieve_of_pRet_pos (R : Retriever S Item) (s0 : S) (U : List Item)
    (hd : Determinate R s0 U) (s : S) (hs : Reachable R s0 U s) (x y : Item)
    (h : 0 < pRet R s x y) : R.retrieve s x = Dist.pure (some y)
```

`pRet R s x y`: the chance that probe `x` retrieves `y`. `Reachable`: reached from `s0` by studying items of `U`.

In words: Studied items retrieve themselves

Theorem. With certainty and stability, after studying a list with no repeats, every item on the list retrieves itself with probability 1.

Proof. Take an item y on the list, and split the list there: the items before y, then y, then the items after.

- Just after y is studied, certainty gives probability 1.
- Every later item differs from y, since there are no repeats. By stability, each later study leaves the probability at 1 (by induction, one item at a time).

Studied items retrieve themselves

```
theorem studied_selfRetrieval (R : Retriever S Item) (s0 : S) (U : List Item)
    (hc : Certainty R s0 U) (hs : Stability R s0 U) (xs : List Item)
    (hU : ∀ x ∈ xs, x ∈ U) (hnd : xs.Nodup) (y : Item) (hy : y ∈ xs) :
    pRet R (R.studyAll s0 xs) y y = 1 := by
  obtain ⟨pre, post, hsplit⟩ := List.append_of_mem hy    -- xs = pre ++ y :: post
  -- ... y ∉ post, from Nodup
  rw [hsplit, show pre ++ y :: post = (pre ++ [y]) ++ post by simp,
    stable_after R s0 U hs y hyU post (pre ++ [y]) _ hpost,   -- studying post changes
    ↪  nothing
    studyAll_append]
  exact hc _ ⟨pre, hpre, rfl⟩ y hyU                          -- certainty, just after y
```

`stable_after` is an induction on `post`, applying stability once per item.

In words: Confident errors from laws

Theorem. Any memory with determinacy, certainty and stability produces confident errors. After studying a list with no repeats that includes q, if a stranger x can be taken for q (with positive probability), the three conclusions of confident errors hold.

Proof.

1. The memory's state is reachable, so the laws apply to it.
2. q retrieves itself with probability 1 (the previous theorem), hence for certain (determinacy).
3. x retrieves q with positive probability, hence for certain (determinacy).
4. These are exactly the hypotheses of the confident-errors theorem.

Confident errors from laws

```
theorem confident_errors_of_laws (R : Retriever S Item) (s0 : S) (U : List Item)
    (hdet : Determinate R s0 U) (hcert : Certainty R s0 U) (hstab : Stability R s0 U)
    (xs : List Item) (hxs : ∀ y ∈ xs, y ∈ U) (hnd : xs.Nodup)
    (correct : P → A) (x q : P) (hq : code q ∈ xs)
    (hconf : 0 < pRet R (R.studyAll s0 xs) (code x) (code q))
    -- ... totals, backups, sharpness, correct x ≠ correct q
    := by
  have hr : Reachable R s0 U (R.studyAll s0 xs) := ⟨xs, hxs, rfl⟩
  have hq' := retrieve_of_pRet_one R s0 U hdet _ hr (code q) (code q)
    (studied_selfRetrieval R s0 U hcert hstab xs hxs hnd (code q) hq)
  have hx' := retrieve_of_pRet_pos R s0 U hdet _ hr (code x) (code q) hconf
  exact confident_errors code _ str as cc n backup correct x q hT hbx hbq hq' hx' hs hne
```

Laws supply the two certain-retrieval hypotheses; the composition theorem does the rest.

In words: Using it: confident misnaming

Face naming. Ann's and Bob's faces have been studied; Carl is a stranger.

- EPAM's net has determinacy, certainty (the 3-bit faces can all be told apart) and stability: three results from the matrix.
- One fact is computed: Carl's face can be taken for Bob's.
- So, for every search length:
  - Bob is never misnamed;
  - every name Carl is given is wrong;
  - Carl says "don't know" exactly as rarely as Bob.

Using it: confident misnaming

```
theorem epam_confident_misnaming_of_laws (n : Nat) : ... :=
  RR.confident_errors_of_laws Person.face str names (1 / 2) n backup (netRetriever F3)
  ↪  (Net.leaf none)
    U3 (Discrimination.determinate F3 U3) (Discrimination.certainty F3 U3 noticeable_U3)
    (Discrimination.stability F3 U3) [ann.face, bob.face] (by decide) (by decide)
    ↪  Person.name carl bob
    (by decide) (by decide +kernel) total_ne (mass_backup carl) (mass_backup bob)
    ↪  sharp_bob
    (by decide)
```

- Three matrix cells for EPAM, cited by name.
- One computed fact, closed by `decide +kernel`: Carl's face can be taken for Bob's.
- No proof script: the theorem is a term, an application of the general result.

In words: Computing inside the proof checker

- Some facts are about particular numbers. For example, with these parameters, $150 + 100$ is answered 200 with probability $45/46$.
- These are proved by **evaluation**: Lean's kernel runs the definitions and checks the resulting equation.
- The same definitions also run as ordinary programs. They produced the exact distributions that the blind simulation was checked against.
- The "lacks" cells of the matrix are proved this way, on small examples.

Computing inside the proof checker

```
theorem example_values :
    prob (trial bcd net str answers (1 / 2) 1 backup x) (wholeSuch (· == 200)) = 45 / 46 ∧
      prob (trial bcd net str answers (1 / 2) 1 backup x) (fun o => o.2 == 250) = 1 / 46
      ↪  := by
  decide +kernel
```

- `decide +kernel`: the kernel evaluates the definitions and checks the equation. Plain `decide` gets stuck on `Rat`.
- The same definitions run under `#eval`: the exported exact distributions that the blind simulation was checked against.
- Counterexample cells (“lacks”) are proved this way, on the eight 3-bit codes.

In words: What is checked, and what is not

- Every theorem is fully proved. Lean reports which axioms a theorem depends on; here, only its three standard ones.
- What proof cannot check is whether the definitions say what EPAM, DOA or ACT-R say. That needs other kinds of evidence: a prose specification, an independent implementation, comparison with community code.

What is checked, and what is not

```
#print axioms Cog.RR.confident_errors_of_laws
'Cog.RR.confident_errors_of_laws' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

- No `sorry` anywhere; only Lean's three standard axioms.
- Core Lean only: no Mathlib, so everything computes.
- **Not checked**: that the definitions say what EPAM, DOA or ACT-R say. That is what the prose specification, the blind simulation, and (planned) comparison with PyACTUp are for.